CAPSTONE PROJECT — PHASE I

PROJECT PROPOSAL & SYSTEM DESIGN

MINI PROJECT: QUIZ APP

An Android-Based Multiple-Choice Quiz Application Using Kotlin

Course Code & Name	ENSP465 — New Age Programming Languages Lab
Submitted by	[Kirti , Rohan , Vaibhav vaid]
Roll Number	[2301010410 , 2301010402 , 2301010289]
Semester / Academic Year	VII / 2026–2027
Section	[E , F]
Submission Date	24 October 2026
Faculty Name	Dr. Hemraj Kumawat
Institution	School of Engineering & Technology, KRMU

Phase I: Project Proposal & System Design

SECTION 2: PROBLEM STATEMENT & MOTIVATION

2.1 Problem Statement

Learning and testing knowledge are important parts of education. Students often need a simple and interesting way to practise questions and check their knowledge of different subjects. Traditional methods, such as solving questions in notebooks or checking answers manually, can take time and may not provide immediate feedback. Students may also find it difficult to track their scores and identify their mistakes during practice.

The proposed Quiz App aims to solve this problem by providing a simple Android application where users can answer multiple-choice questions across different subjects. Users will be able to choose a subject, such as Computer Science, Mathematics, Science, or General Knowledge, and then select a difficulty level: Easy, Medium, or Hard. The application will display questions with multiple answer options, check the selected answer, and provide instant feedback. It will calculate the user's score and show the final result after the quiz is completed.

2.2 Real-World Context

The project belongs to the educational technology domain and is designed as a mobile application for Android devices. It can be useful for students who want to practise subjects such as Computer Science, Mathematics, Science, and General Knowledge. Each subject will offer Easy, Medium, and Hard difficulty levels so beginners can start with simpler questions and more confident users can attempt challenging ones. The application will include a timer to make quizzes more engaging and help users practise answering questions within a limited time.

2.3 Motivation

The main motivation behind this project is to make learning easier, faster, and more interactive. Immediate feedback can help users understand whether their answers are correct or incorrect. Score calculation allows users to evaluate their performance, while the timer adds a time-management element to the quiz. This project will also provide practical experience in Kotlin programming, Android application development, user interface design, data handling, and database management.

2.4 Existing Solutions and Their Limitations

Several online quiz platforms and educational applications already provide quizzes and practice tests. However, some solutions may require an internet connection, contain many additional features, or have interfaces that are not suitable for users who only want a simple practice quiz. The proposed

Quiz App will focus on the basic requirements of a quiz system, including multiple-choice questions, instant feedback, score calculation, a timer, and question storage. It will aim to provide a simple and easy-to-use interface for students.

SECTION 3: PROPOSED SOLUTION & LANGUAGE JUSTIFICATION

3.1 Project Overview

The Quiz App will be an Android-based application developed using Kotlin. Before starting a quiz, users will choose a subject (for example, Computer Science, Mathematics, Science, or General Knowledge) and a difficulty level (Easy, Medium, or Hard). The app will then load questions matching those choices. Users will view multiple-choice questions, select answers, and receive feedback on their responses. The application will maintain the user's score, manage the quiz timer, and display a final result after all questions have been answered or the available time has ended. Questions, answer options, subject names, and difficulty levels will be stored in a database.

3.2 Programming Language Chosen: Kotlin

Kotlin is selected as the primary programming language because it is suitable for Android application development and supports clear, concise, and reliable code.

3.3 Reasons for Choosing Kotlin

Android Application Development: Kotlin is officially supported for Android development. It provides the features required to build Android screens, handle user interactions, and implement application logic.

Null Safety: Kotlin includes null-safety features that help reduce common errors related to null values. This is useful when handling question records, selected answers, and database results.

Data Classes: Kotlin data classes can represent quiz questions, answer options, and quiz results in a structured way. They make it easier to store and manage related information.

Coroutines and Asynchronous Operations: Kotlin coroutines can be used for operations such as retrieving questions from a database without blocking the main user interface. This helps keep the application responsive.

Readable and Concise Code: Kotlin reduces unnecessary code and provides useful language features such as functions, collections, and extension functions. These features can make the quiz logic easier to develop, understand, and maintain.

3.4 Proposed Technologies and Libraries

- Programming Language: Kotlin
- Development Environment: Android Studio
- User Interface: Android Views with XML layouts, or Jetpack Compose if selected during implementation
- Database: Room Persistence Library with SQLite
- Asynchronous Processing: Kotlin Coroutines, where required
- Platform: Android

The final choice of user-interface approach will be confirmed during implementation. Room will be used to provide structured local storage for quiz questions and related information.

SECTION 4: OBJECTIVES & SCOPE

4.1 Objectives

1. To develop an Android application that provides multiple-choice quizzes for different subjects, including Computer Science, Mathematics, Science, and General Knowledge.
2. To provide three difficulty levels—Easy, Medium, and Hard—so users can select a quiz suited to their knowledge and confidence.
3. To provide at least three or four answer options for each question.
4. To check selected answers and provide immediate feedback on correctness.
5. To calculate the user's score and display the final result at the end of the quiz.
6. To implement a timer and store quiz questions in a database for retrieval during the quiz.

4.2 In-Scope Features

- A home screen where users can choose a subject and difficulty level before starting a quiz.
- Multiple-choice questions with answer options, filtered by the selected subject and difficulty level.
- Question navigation and answer selection.
- Instant feedback for correct and incorrect answers.
- Automatic score calculation.
- A countdown timer for the quiz.
- Local database storage for questions, options, correct answers, subject names, and difficulty levels.

- A result screen showing the total score and performance.
- Basic input validation and testing.

4.3 Out-of-Scope Features

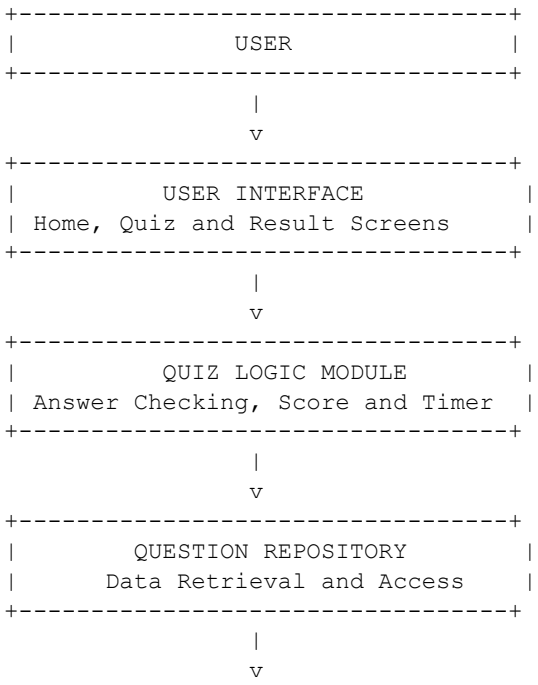
- Online multiplayer quizzes.
- Live competitions between multiple users.
- User accounts, login, and social sign-in.
- Cloud-based synchronisation.
- Online leaderboards.
- Payment or subscription features.
- Artificial intelligence-based question generation.

These features may be considered for future development but are not part of the current project scope.

SECTION 5: SYSTEM ARCHITECTURE & DESIGN

5.1 High-Level System Architecture

The application will follow a simple modular architecture. The user interface will handle user interactions, including subject and difficulty selection. The quiz logic module will load matching questions and manage scoring and timing, while the repository/database layer will retrieve and store quiz data.



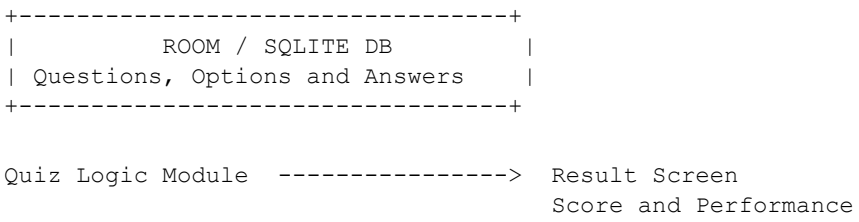


Figure 1: High-Level Architecture of the Quiz App

5.2 Component Descriptions

User Interface Module. This module provides the screens through which users interact with the application. It will contain the home screen, subject selection, difficulty selection, quiz screen, answer options, timer display, and result screen. It accepts user actions, such as choosing a subject and level, starting the quiz, and selecting an answer, and displays the appropriate information.

Quiz Logic Module. This module controls the main quiz operations. It manages the current question, checks whether the selected answer is correct, updates the score, and tracks question progress. It also manages the timer and determines when the quiz should end.

Question Repository Module. The repository module acts as an intermediary between the quiz logic and the database. It retrieves questions that match the user's selected subject and difficulty level and provides the question data to the application. This separation makes the application easier to maintain.

Database Module. The database will use Room with SQLite for local storage. It will store question text, answer options, correct answers, subject/topic information, and difficulty level. When a quiz begins, the application will retrieve questions matching the selected subject and level.

Result Module. The result module displays the final score after the quiz ends. It will show the number of correct answers and the total number of questions. The result will be calculated using the answers submitted by the user.

5.3 Data Flow Diagram — Level 0

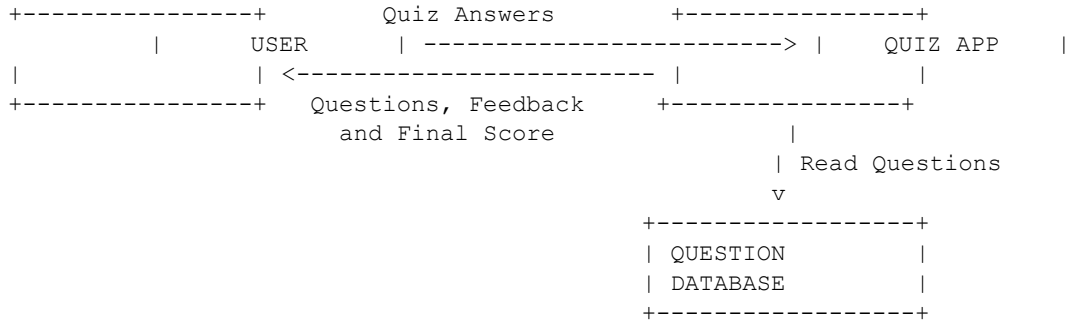


Figure 2: Level 0 Data Flow Diagram

The user starts the quiz and receives questions from the application. The user selects answers, and the application checks them against the correct answers stored in the question database. It provides feedback, updates the score, and displays the final result.

5.4 Main Workflow / Sequence Diagram

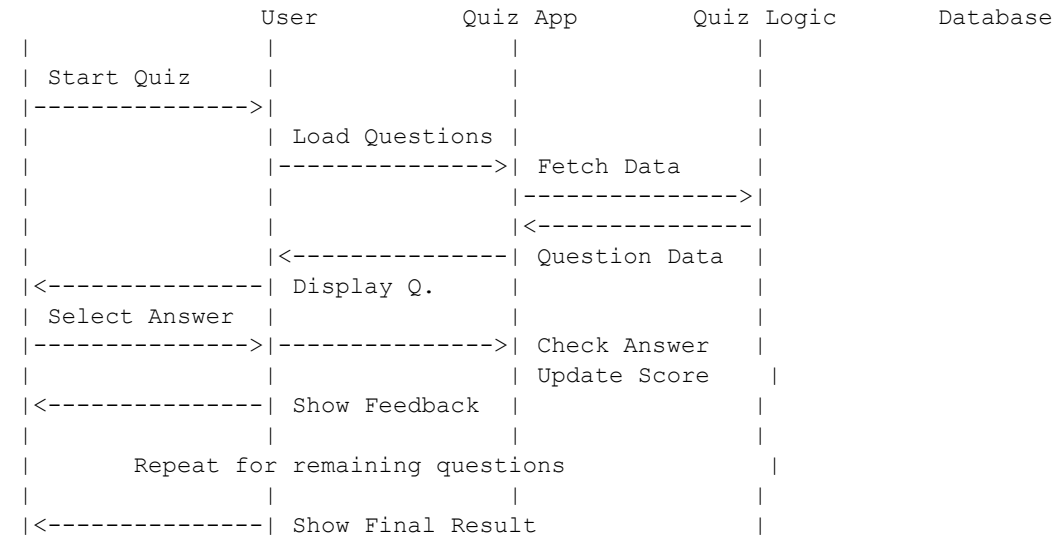


Figure 3: Sequence Diagram of Quiz Execution

5.5 Input-Process-Output (IPO) Description

Input

- User's selected subject and difficulty level, followed by the action to start the quiz.
- Selected answer for each question.
- Questions and answer options retrieved from the database.
- Timer duration configured for the quiz.

Process

7. Retrieve questions from the database that match the selected subject and difficulty level.
8. Display each question with its answer options.
9. Accept the user's selected answer.
10. Compare the selected answer with the correct answer.
11. Update the score for correct answers.
12. Update the question progress and timer.
13. End the quiz when all questions are completed or the timer expires.
14. Calculate and display the final result.

Output

- Questions and answer options displayed on the screen.
- Instant feedback about answer correctness.
- Updated score and question progress.
- A final result showing the score and correct answers.

5.6 Key Data Structures and Kotlin Features

A Kotlin data class will represent each question. It will contain a unique ID, question text, subject, difficulty level, answer options, and the correct answer.

```
data class Question(
    val id: Int,
    val questionText: String,
    val topic: String,
    val options: List<String>,
    val correctAnswerIndex: Int
)
```

A separate data class can represent the final result.

```
data class QuizResult(
    val totalQuestions: Int,
    val correctAnswers: Int,
    val score: Int
)
```

Score Calculation

The application will increase the score when the user's selected answer matches the correct answer.

```
if (selectedAnswerIndex == question.correctAnswerIndex) {
    score++
}
```

The final score can be represented as the number of correct answers out of the total questions. A percentage can also be calculated as: Score Percentage = (Correct Answers / Total Questions) × 100. The application must handle the case where the total number of questions is zero to avoid division by zero.

Database Design

Field	Data Type	Description
id	Integer	Unique question ID
questionText	Text	Question statement
subject	Text	Subject/category of the question, such as Mathematics or Science
difficulty	Text	Difficulty level of the question: Easy, Medium, or Hard

optionA	Text	First answer option
optionB	Text	Second answer option
optionC	Text	Third answer option
optionD	Text	Fourth answer option
correctAnswerIndex	Integer	Index of the correct option

Room entities, DAOs, and a database class will be used to define the database structure and perform database operations.

Subject and Difficulty Selection

The home screen will allow the user to choose a subject such as Computer Science, Mathematics, Science, or General Knowledge. After selecting a subject, the user will choose Easy, Medium, or Hard. The application will filter the question bank using both selections so that the quiz contains relevant questions at the chosen level. For example, an Easy Computer Science quiz will show introductory computer science questions, while a Hard Mathematics quiz will show more challenging mathematics questions.

Timer Management

The quiz will use a countdown timer. The remaining time will be updated on the quiz screen. When the timer reaches zero, the quiz will end and the result screen will be displayed. Timer callbacks will be handled carefully so that they do not continue running after the quiz has ended.

SECTION 6: WORK PLAN & TIMELINE

The project will be completed in milestones. The tentative schedule below may be adjusted according to the faculty's Phase II and Phase III submission dates.

Milestone	Tentative Period	Planned Work / Deliverable
1. Proposal and Design	22–24 October 2026	Phase I report, requirements, diagrams, and proposed architecture
2. Interface Development	26 October–1 November 2026	Home screen, quiz screen, answer options, and result screen
3. Quiz Logic and Timer	2–8 November 2026	Answer checking, score calculation, question navigation, and timer
4. Database Integration	9–15 November 2026	Room/SQLite database, question storage, and retrieval
5. Integration and Testing	16–22 November 2026	Integrated application, functional testing, and bug fixing
6. Final Documentation and Viva Preparation	23–29 November 2026	Final report, screenshots, test results, demonstration, and viva preparation

Phase II Deliverables

A working intermediate version of the Android application, including the main interface, quiz functionality, timer, score calculation, and database integration.

Phase III Deliverables

The completed application, testing results, final project documentation, demonstration, and preparation for the viva.

Task Distribution

If the project is completed individually, all tasks—including design, coding, database integration, testing, and documentation—will be performed by the student. For a team project, responsibilities will be divided among members according to the agreed module allocation.

SECTION 7: REFERENCES

[1].JetBrains, “Kotlin Documentation.” Available: https://kotlinlang.org/docs/home.html. Accessed: October 2026.

[2].Android Developers, “Kotlin for Android.” Available: https://developer.android.com/kotlin. Accessed: October 2026.

[3].Android Developers, “Save data in a local database using Room.” Available: https://developer.android.com/training/data-storage/room. Accessed: October 2026.

[4].Android Developers, “Guide to app architecture.” Available: https://developer.android.com/topic/architecture. Accessed: October 2026.

— End of Phase I Project Proposal & System Design —